

Musterlösung — Übungsklausur 8

Deep Learning 1 · TU Berlin

1 Multiple Choice

1. Cross-Entropy vs. Log-Loss binär?

✓ (b) Äquivalent — Log-Loss ist Cross-Entropy für binären Fall (bewiesen via Softmax-Mapping)

- (a) Verschiedene Aufgaben
- (c) Perplexity vs. Accuracy
- (d) Log-Loss nur SVM

2. Seq2Seq RNN Architekturen?

✓ (b) Standard, Bidirektional und Encoder-Decoder RNNs

- (a) Nur Standard
- (c) Nur Encoder-Decoder
- (d) Nur Bidirektional

3. Conv2d(3,20,5) Parameter?

✓ (a) 1520 ($= 20 \times 3 \times 5 \times 5 + 20 = 1500 + 20$)

- (b) 1500
- (c) 320
- (d) 1505

4. Topological Sparse Coding?

✓ (b) Quelldimensionen müssen auf einem 2D-Gitter korrelieren

- (a) Hierarchische Strukturen
- (c) CNN Feature Maps
- (d) L2-Norm

5. $\tanh(0)$?

✓ (b) 0 ($\tanh(0) = \sinh(0)/\cosh(0) = 0/1 = 0$)

- (a) 1
- (c) 0.5
- (d) -1

6. Encoder-Decoder RNN?

✓ (b) Eingangssequenz $\rightarrow$ Repräsentation $s \rightarrow$ Ausgangssequenz aus s

- (a) Encoder verarbeitet Output
- (c) Keine geteilten Parameter
- (d) Nur feste Längen

7. Texture Bias?

✓ (b) CNNs lernen bevorzugt Texturfeatures statt Formfeatures

- (a) Können keine Texturen lernen
- (c) Invariant
- (d) Kein reales Problem

8. Dataset Bias?

✓ (b) Überrepräsentation bestimmter Verteilungsbereiche $\rightarrow$ falsche Annahmen über Gesamtverteilung

- (a) Gewichts-Bias
- (c) Kleines Trainingsset

2 Theorie — Lernproblem in Deep Learning

(a) Allgemeines Lernproblem

$$E(\theta) = (1/N) \sum_{i=1}^N \ell(f(x_i, \theta), t_i)$$

Trainings-Error: Fehler auf Trainingsdaten. True Error: Erwarteter Fehler unter der wahren Verteilung $p(x,t)$. Wir minimieren Trainings-Error als Proxy für True Error. Gap entsteht durch endliche Daten und Modellannahmen.

(b) Endliche Daten & Overfitting

Endliche Daten: Trainingsdaten sind zufällige Stichprobe aus $p(x,t)$ → lokal dominante Aspekte der Verteilung können das Modell zu Überanpassung verleiten (Modell lernt 'spurious variations').

Spurious Correlations: Task-irrelevante Variablen zufällig korreliert mit relevanten → Modell basiert Entscheidungen darauf → versagt bei Verteilungsänderungen.

(c) Best-Complexity (Vapnik)

Nach Vapnik: optimale Modellkomplexität ist die, die den Test-Fehler minimiert (nicht Trainings-Fehler). Zu einfach → Underfitting. Zu komplex → Overfitting.

Praktisch: Hyperparameter-Suche mit Holdout Validation oder Cross-Validation.

3 Theorie — Autoencoder-Varianten

(a) Linearer Autoencoder & PCA

$$\min_{f,g} (1/N) \sum_i \|x_i - f(g(x_i))\|^2 \text{ mit } f, g \text{ linear}$$

Lösung entspricht PCA: g (Encoder) = Projektion auf k Hauptkomponenten, f (Decoder) = Rückprojektion. Die Autoencoder-Lösung und PCA-Lösung spannen denselben Unterraum auf.

(b) Limitierung linear & Lösung

Problem: Lineare Encoder können keine nichtlinearen Manifolds in den Daten repräsentieren.

Komplexe Strukturen wie Bilder liegen auf nichtlinearen Mannigfaltigkeiten.

Lösung: Aktivierungsfunktionen (z.B. ReLU) in den Encoder einfügen. Decoder von $\max(0, Vx)$ statt Vx .

(c) Topological Sparse Coding

Zusätzliche Einschränkung: Quelldimensionen müssen auf einem vordefinierten 2D-Gitter korrelieren. Benachbarte Quellen sollen ähnliche Aktivierungen haben.

Motivation: Modelliert topographische Struktur wie im visuellen Kortex (räumliche Nachbarschaft von Features hat semantische Bedeutung).

4 Theorie — Shapley Values & LRP

(a) Shapley Values Formel

$$\phi_i = \sum_{S: i \notin S} |S|!(d-|S|-1)!/d! \cdot [f(x_{S \cup \{i\}}) - f(x_S)]$$

S : Subset aller Features ohne Feature i . $f(x_S)$: Modelloutput wenn nur Features in S bekannt. Der Term $[f(x_{S \cup \{i\}}) - f(x_S)]$: Marginalbeitrag von Feature i . Gewichtung: alle möglichen Ordnungen gleichmäßig.

(b) Computationales Problem & Lösung

Problem: Exakte Berechnung erfordert 2^d Feature-Subset-Evaluierungen $\rightarrow$ exponentiell in $d \rightarrow$ für hohe Dimensionen ($d > 20$) nicht praktikabel.

Lösung: Monte-Carlo-Approximation über T zufällig gewählte Permutationen. Shapley-Wert wird als Durchschnitt der Marginalcontributions geschätzt.

(c) LRP vs. Shapley Values

LRP: Sehr schnell (~ 1 Durchlauf). Gut für tiefe Netze. Muss für neue Architekturen angepasst werden. Gut für räumliche Explanations (Heatmaps bei Bildern).

Shapley: Theoriefundiert (fairste Attribution laut Spieltheorie). Modell-agnostisch. Rechenintensiv. Besser wenn Interaktionen zwischen Features wichtig sind.

5 Programmierung — Regularisierung

(a) Weight Decay in PyTorch

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01,
weight_decay=1e-4)
```

weight_decay λ entspricht L2-Regularisierung: $L_{\text{reg}} = L + \lambda \cdot \theta^2$. Gradient: $\partial L_{\text{reg}} / \partial \theta = \partial L / \partial \theta + 2\lambda \cdot \theta$. PyTorch multipliziert weight_decay direkt mit θ bei jeder Update-Schritt (äquivalent).

(b) Manueller Dropout Layer

```
class ManualDropout(nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p
    def forward(self, x):
        if not self.training:
            return x # Beim Testing: keine Dropout
        mask = torch.bernoulli(torch.full_like(x, 1-self.p))
        return x * mask / (1 - self.p) # Inverted Dropout: skaliert
```